Operating Systems

Assignment -9

Question 1 — Printing the Page Table of a Process

xv6 (x86) uses a two-level page table with 4 KB pages. Write a system call that prints the page table of the calling process so that you can see how its address space is mapped.

Tasks:

a) Implement a system call int vmprint(void) that walks the page directory of the calling process (use walkpgdir() and the structure of freevm() in vm.c as a guide). For every valid user page, print its virtual address, the physical address of its frame, and the flags P, W and U. Ignore the kernel part of the address space (addresses at or above KERNBASE).

b) At the end, print the total number of valid user pages and the number of page-table pages used by the process.

c) Write a user program pttest.c that calls vmprint() at start-up and again after calling sbrk(4 * 4096), and show that exactly four new pages appear.

Question 2 — Handling Page Faults (Lazy Allocation)

Currently, sbrk() in xv6 allocates memory immediately, and any page fault kills the process. In this question you will delay allocation until the page is first used, by handling the page fault in the kernel.

Tasks:

a) Modify sys_sbrk() in sysproc.c so that when n > 0 it only increases myproc()->sz and does not allocate memory. Make sure that deallocuvm() and copyuvm() in vm.c skip pages that were never allocated instead of calling panic().

b) In trap.c, handle T_PGFLT: read the faulting address with rcr2(). If it is below myproc()->sz, allocate a zeroed page with kalloc() and memset(), map it with mappages() using PTE_W | PTE_U, count the fault in a new field of struct proc, and resume the process. Otherwise print the pid and address and kill the process.

c) Write a user program faulttest.c that calls sbrk(32 * 4096), writes to exactly 5 different pages, and prints the number of page faults handled (return it through a small system call getfaults()). The result must be 5. Also show that accessing an address above the allocated size kills only that process.

Question 3 — Page Replacement Algorithms (FIFO and LRU)

Write a program that runs on xv6 and compares the FIFO and LRU page replacement algorithms on a given reference string.

Tasks:

a) Write a user program pagerepl.c that takes the number of frames and a reference string as input, and implements FIFO and LRU. For every reference, print the frames and whether it was a hit or a fault.

b) Test it with the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 and 3 frames. The expected results are 15 page faults for FIFO and 12 page faults for LRU.

c) Run both algorithms for 3, 4 and 5 frames on the same string and print a table of the number of page faults and the hit ratio for each.

Common Submission Requirements (for all 3 questions)

a) All modified kernel source files (vm.c, trap.c, proc.h, sysproc.c, syscall.c, syscall.h, defs.h, user.h and usys.S as needed), with every change clearly commented.

b) User test programs: pttest.c, faulttest.c, pagerepl.c.

c) Terminal output screenshots for each question demonstrating correct behavior.

d) A short report (1 page) explaining what you changed for each question and what the output shows.